

Conformance Checking for Process Improvement

JM0211 Process Mining

Group 8 — Rick de Rijk, Thom Verzanvoort, Stefan Vonk, Tycho van Rooij

Table of contents

1 Motivation and Scientific Background	2
2 Conceptual Approach	2
2.1 Overview	2
2.2 Step 1 KPI definition and enrichment	2
2.3 Step 2 Alignment and encoding	3
2.4 Step 3 Classification	3
2.5 Step 4 Rules and log split	3
3 Implementation	3
4 Evaluation Design and Results	3
5 Limitations and Future Work	3
References	4

List of Figures

i Working draft

Page limit on submission: **5 pages excluding references**. Remove this callout before the final PDF. Section ownership while drafting: Rick = KPI / data step; others fill alignment, classification, evaluation.

1 Motivation and Scientific Background

Conformance checking compares behaviour allowed by a normative process model with behaviour recorded in an event log (Song and Aalst 2008; Aalst 2016). Diagnostics can expose control-flow deviations (for example skipped or inserted activities) and, when data attributes are available, violations of data guards. Those discrepancies are often treated as defects. Prior work, however, shows that some deviations are workarounds that coincide with *better* process performance (Dees, Leoni, and Mannhardt 2017). Recognising which deviations help and which hurt is therefore a concrete route to process improvement, not only to compliance reporting.

This assignment implements the **deviation analysis** step of the methodology of Dees, de Leoni and Mannhardt (Dees, Leoni, and Mannhardt 2017): correlate observed deviations with a chosen key performance indicator (KPI), then retain rules that link deviation patterns to KPI outcomes. In the JM0211 group topics, our focus sits with conformance checking (shared with several other groups) but is aimed at *improvement* rather than at resource or temporal conformance alone. Predictive and prescriptive monitoring groups also care about performance; we differ in that we start from alignments against a Petri net and explain KPI variation through deviation features, instead of forecasting the next event or prescribing an action.

The business setting for evaluation is the BPI Challenge 2017 loan-application log (Denied subset) with a provided Petri net. Throughput time of a case is a natural KPI for that domain: faster handling of applications is valuable for both the applicant and the organisation, and it matches the KPI used in the UWV case study of (Dees, Leoni, and Mannhardt 2017).

2 Conceptual Approach

2.1 Overview

The artefact takes as input (i) an event log in XES format and (ii) a Petri net in PNML format. It returns (i) a set of classification rules that relate deviation encodings to the KPI, and (ii) for each rule, the subset of traces that satisfy it. The pipeline has four steps.



2.2 Step 1 KPI definition and enrichment

Owner: Rick (Person A).

KPI. We use *throughput time* of each trace, following (Dees, Leoni, and Mannhardt 2017): for a trace σ ,

$$\kappa(\sigma) = t(\text{last event of } \sigma) - t(\text{first event of } \sigma).$$

We store $\kappa(\sigma)$ in **days** (elapsed seconds divided by 86400) as a case-level attribute named `throughput_time`. Lower values are better. Empty traces are rejected; a missing `time:timestamp` on an event is treated as an error. A single-event case receives throughput 0.

Required log characteristics. The log must provide a clear case notion and an ordered sequence of events with reliable timestamps. Control-flow activity labels are required for later alignment; resources are not required for this KPI step. The Petri net must be the normative model against which alignments will be computed in the next step.

Output of this step. The same event log, enriched with `throughput_time` on every case (optionally exported as a new XES file). No parameters beyond optional subsampling for development speed.

2.3 Step 2 Alignment and encoding

Owner: Person B (to draft).

For each trace, compute an optimal alignment with the Petri net. Encode each trace by counting, per activity a_i , the number of model moves and the number of log moves. This yields two features per activity (Dees, Leoni, and Mannhardt 2017).

2.4 Step 3 Classification

Owner: Person C (to draft).

Train a classification tree on the encoded traces, using `throughput_time` as the dependent variable. Use standard hyperparameter search to select a well-performing tree. *Note:* if the KPI is kept continuous, a regression tree may be the direct analogue of (Dees, Leoni, and Mannhardt 2017); if the course brief is read strictly as a *classification* tree, the KPI must be discretised (for example by a threshold). The group should lock that choice before modelling.

2.5 Step 4 Rules and log split

Owner: Person D (to draft).

Derive classification rules from the tree and split the original log so that each rule maps to a compliant subset of traces. These subsets are the second required artefact output.

3 Implementation

Person A (done in code). Python 3 with `pm4py` loads the XES log and PNML net. A small module computes and attaches `throughput_time`, with a CLI entry point (`main.py`) supporting `--sample` and optional `--export` of an enriched XES. Paths are relative to the project root.

Persons B–D. To be filled: alignment API, encoding schema, scikit-learn tree setup, rule extraction, and how to reproduce evaluation scores from the repository README.

4 Evaluation Design and Results

Owner: Person D (lead), with group input.

The brief requires a manually built gold standard and at least precision and recall for (1) optimal alignments per trace and (2) the recovered classification rules. Dataset: BPI Challenge 2017 (Denied) as provided with the assignment; a sample may be used during development, with final numbers on the full log.

Results and figures: pending implementation of steps 2–4.

5 Limitations and Future Work

KPI step. Throughput time uses only the first and last timestamps. It ignores waiting-time structure inside the case, calendar effects, and unfinished cases that might appear in a broader log. Discretising the KPI for a classification tree (if required) throws away magnitude information that a regression tree would keep (Dees, Leoni, and Mannhardt 2017).

Broader approach. Further limitations of alignments, tree stability, and gold-standard construction will be added once those stages exist. Future work suggested by (Dees, Leoni, and Mannhardt 2017) includes multi-KPI improvement and incorporating only legally admissible repairs; those sit outside our implemented deviation-analysis scope but remain relevant for process improvement.

References

- Aalst, Wil M. P. van der. 2016. *Process Mining: Data Science in Action*. 2nd ed. Springer. <https://doi.org/10.1007/978-3-662-49851-4>.
- Dees, Marcus, Massimiliano de Leoni, and Felix Mannhardt. 2017. “Enhancing Process Models to Improve Business Performance: A Methodology and Case Studies.” In *On the Move to Meaningful Internet Systems. OTM 2017 Conferences*, 3–22. Springer.
- Song, Minseok, and Wil M. P. van der Aalst. 2008. “Towards Comprehensive Support for Organizational Mining.” *Decision Support Systems* 46 (1): 300–317.